

UNIVERSIDAD DE LAS PALMAS DE GRAN CANARIA

Escuela de Ingeniería Informática

Práctica 7:
SERVIDOR WEB SEGURO CON
CERTIFICADOS DE CLIENTE

Asignatura: Seguridad de la Información

Curso: 4º de Grado en Ingeniería Informática

Autor:

Nicolás Rey Alonso

`nicolas.rey101@alu.ulpgc.es`

Fecha: Abril 2026

Índice general

1. Introducción	3
1.1. Requisitos Previos	3
1.2. Objetivos de la práctica	3
1.3. Fundamentos teóricos	4
1.4. Enunciado de la práctica	4
2. Incorporación de una segunda Autoridad de Certificación (FNMT)	6
2.1. Enunciado	6
2.2. Resolución	6
2.2.1. Descarga del certificado AC Raíz FNMT-RCM	6
2.2.2. Verificación de la huella digital (SHA-1)	7
2.2.3. Conversión de DER a PEM	7
2.2.4. Generación del fichero de CAs confiables	8
3. Configuración del servidor Apache	10
3.1. Enunciado	10
3.2. Resolución	10
4. Instalación del script de sustitución de <code>index.html</code>	13
4.1. Enunciado	13
4.2. Resolución	13
5. Rearranque del Apache y verificación del <code>error_log</code>	15
5.1. Enunciado	15

5.2. Resolución	15
6. Conexión de prueba con openssl s_client	17
6.1. Enunciado	17
6.2. Resolución	17
7. Conexión desde Firefox con el certificado de la asignatura	19
7.1. Enunciado	19
7.2. Resolución	19
8. Conexión desde Firefox con el segundo certificado de cliente	22
8.1. Enunciado	22
8.2. Resolución	22
9. Conclusiones	24

Capítulo 1

Introducción

1.1. Requisitos Previos

- Servidor web Apache instalado y configurado.
- Certificados de cliente necesarios: certificado de la autoridad de la asignatura y un certificado secundario (ej. FNMT).
- Herramientas OpenSSL disponibles.

1.2. Objetivos de la práctica

- Modificar la configuración del servidor web Apache para que acepte certificados de cliente firmados por la misma autoridad de certificación y por una autoridad adicional (preferentemente FNMT).
- Habilitar la ejecución de scripts CGI/binarios (bash) o Python para interactuar con los datos del certificado.
- Instalar un script que reemplace el fichero `index.html` para mostrar los datos más importantes del certificado presentado por el cliente (Common Name, Issuer, Validity, etc.) a través de variables de entorno.
- Demostrar la capacidad del servidor web para aceptar dos conexiones distintas utilizando certificados firmados por autoridades diferentes.
- Verificar la correcta comunicación y aceptación de certificados mediante comandos `openssl s_client` y navegadores (Firefox), incluyendo la visualización de los datos del certificado en el script.

1.3. Fundamentos teóricos

Para la realización de esta práctica, se aplican diversos conceptos teóricos clave de la ciberseguridad y la gestión de redes:

- **Certificados Digitales y Autoridades de Certificación (CA):** Entender cómo funcionan los certificados X.509, la estructura de una cadena de confianza y el rol de las Autoridades de Certificación (como FNMT) en la emisión y validación de identidades digitales.
- **Configuración del Servidor Web SSL/TLS:** Comprender cómo configurar un servidor web (Apache) para que gestione la autenticación de clientes mediante certificados (usando módulos como `mod_ssl`) y cómo gestionar múltiples CAs confiables.
- **Validación de Certificados en el Handshake TLS:** Analizar el proceso de negociación TLS, donde el cliente presenta su certificado al servidor. Esto implica verificar la validez del certificado del cliente y asegurar que la cadena de confianza (la ruta de la CA) es válida para establecer una conexión segura.
- **Interacción con el Sistema Operativo (Scripts CGI/Python):** Entender cómo los scripts pueden interactuar con el sistema operativo y extraer metadatos específicos (como Common Name, Issuer, Validity) contenidos dentro del certificado digital para presentarlos al usuario o al servidor web.
- **Mecanismos de Autenticación Múltiple:** Demostrar la capacidad del servidor para validar certificados emitidos por diferentes CAs simultáneamente, lo que requiere una configuración adecuada en el servidor para manejar múltiples *trust stores*.
- **Herramientas de Diagnóstico (OpenSSL y Navegadores):** Utilizar herramientas como `openssl s_client` para verificar directamente la comunicación SSL/TLS, la presencia de certificados y la correcta validación de las firmas entre cliente y servidor.

1.4. Enunciado de la práctica

El desarrollo del ejercicio y de la presente memoria seguirán el guion de actividades propuesto en el enunciado:

1. Modificación de la configuración del Apache

- Configurar el servidor para que solicite certificados de cliente y acepte los firmados por la misma autoridad de la asignatura y por otra autoridad (preferentemente FNMT).

- Habilitar la ejecución de scripts CGI-bin (`bash`), código Python, etc.

2. Instalación del script de sustitución de `index.html`

- Breve explicación del script utilizado y visualización de sus permisos.

3. Carga del segundo certificado de cliente en el Firefox

- Preferentemente, utilizar un certificado FNMT real. Alternativamente, crear una autoridad de certificación nueva, generar un segundo certificado de cliente para el usuario con esta nueva autoridad e instalarlo en el Firefox, junto con el certificado de la autoridad.
- Si se utiliza el certificado FNMT, los certificados de autoridad ya se encuentran en el directorio del sistema Linux (o no) y deben incorporarse a la configuración del Apache.
- Si se utiliza otra autoridad, repetir los pasos para su certificado raíz y asegurarse de que se utiliza la extensión `-addext "basicConstraints=CA:TRUE"` en el comando de creación del certificado autofirmado de autoridad.

4. Rearranque del Apache y verificación del fichero `error_log`

- Realizar el reinicio del servicio Apache.
- Verificar el contenido del fichero `error_log`.

5. Conexión de prueba con el comando `openssl s_client` libre de errores

- Ejecutar el siguiente comando para verificar la conexión y los certificados:

```
openssl s_client www.ejemplo.com:443 -cert certcliente.pem -key c
```

- Verificar que no se producen errores en la conexión ni en los certificados.
- Verificar que NO hay errores en `error_log`.

6. Conexión sin errores desde Firefox con el servidor, con el certificado de cliente de la autoridad de la asignatura

- Asegurar que el script muestre los datos más importantes del certificado de cliente.
- Mostrar captura de pantalla de la conexión (ejecución del script).

7. Conexión sin errores desde Firefox con el servidor, con el certificado de cliente FNMT o uno secundario creado por el estudiante

- Asegurar que el script muestre los datos más importantes del certificado de cliente.
- Mostrar captura de pantalla de la conexión (ejecución del script).

Capítulo 2

Incorporación de una segunda Autoridad de Certificación (FNMT)

2.1. Enunciado

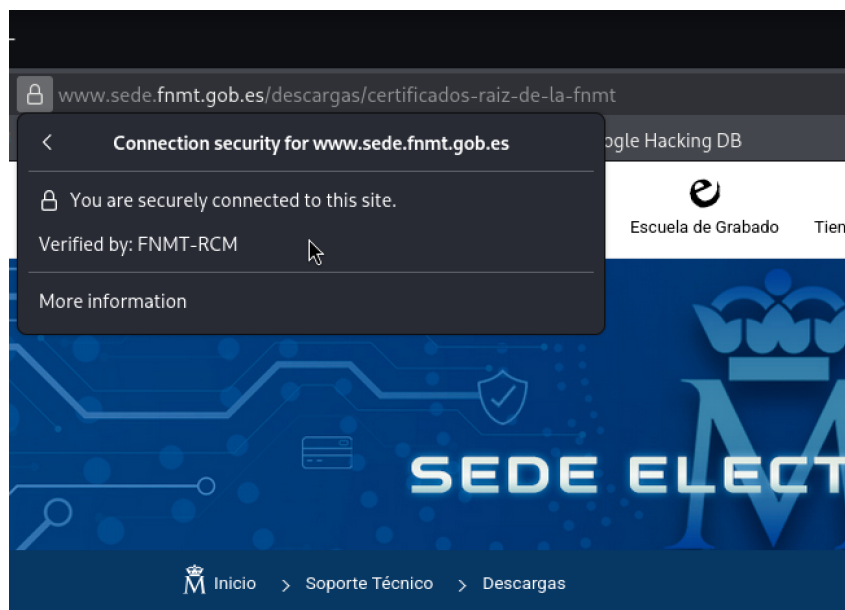
Carga del segundo certificado de cliente en Firefox. Preferentemente, un certificado FNMT real. Alternativamente, crear una autoridad de certificación nueva, crear un segundo certificado de cliente para el usuario con dicha autoridad e instalarlo en el Firefox. Si se utiliza certificado FNMT, los certificados de autoridad ya se encuentran en el directorio de la máquina Linux (o no) y se deben incorporar a la configuración del Apache.

2.2. Resolución

Para que el servidor Apache acepte certificados de cliente firmados por una segunda autoridad, se ha optado por incorporar la **AC Raíz FNMT-RCM** como Autoridad de Certificación adicional, junto con la autoridad de la asignatura.

2.2.1. Descarga del certificado AC Raíz FNMT-RCM

Se accede a la sede electrónica de la FNMT (<https://www.sede.fnmt.gob.es>) y se descarga el certificado de la **AC Raíz FNMT-RCM** en formato DER (`AC_Raiz_FNMT-RCM_SHA256.cer`).



Certificados AC Raíz de la FNMT

Figura 2.1: Página oficial de descarga de la AC Raíz FNMT-RCM.

2.2.2. Verificación de la huella digital (SHA-1)

Antes de incorporar el certificado al sistema, se comprueba la autenticidad del fichero descargado contrastando su huella SHA-1 contra la publicada por la FNMT en su web oficial.

```
1 openssl dgst -sha1 AC_Raiz_FNMT-RCM_SHA256.cer
2 diff <(echo ec503507b215c4956219e2a89a5b42992c4c2c20) \
3     <(echo ec503507b215c4956219e2a89a5b42992c4c2c20)
```

Al no devolver el comando `diff` ninguna diferencia, queda verificado que el certificado descargado es íntegro y se corresponde con el publicado por la FNMT.

Descarga certificado AC Raiz FNMT-RCM
 HUELLA SHA-1: `ec503507b215c4956219e2a89a5b42992c4c2c20`
Certificados subordinados
 Descarga certificado AC FNMT Usuarios

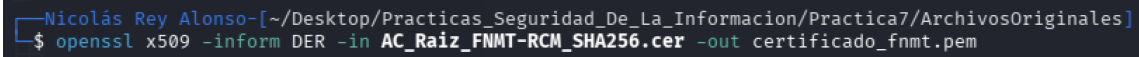
```
Session Actions Edit View Help
Nicolás Rey Alonso[~/Downloads]
$ openssl dgst -sha1 AC_Raiz_FNMT-RCM_SHA256.cer
SHA1(AC_Raiz_FNMT-RCM_SHA256.cer)= ec503507b215c4956219e2a89a5b42992c4c2c20
$ diff <(echo ec503507b215c4956219e2a89a5b42992c4c2c20) <(echo ec503507b215c4956219e2a89a5b42992c4c2c20)
$
```

Figura 2.2: Comparación de la huella SHA-1 publicada con la calculada localmente.

2.2.3. Conversión de DER a PEM

El certificado se descarga en formato binario DER. Apache requiere los certificados en formato PEM (Base64), por lo que se realiza la conversión con OpenSSL:


```
1 openssl x509 -inform DER -in AC_Raiz_FNMT-RCM_SHA256.cer \  
2 -out certificado_fnmt.pem
```



```
Nicolás Rey Alonso-[~/Desktop/Practicas_Seguridad_De_La_Informacion/Practica7/ArchivosOriginales]  
$ openssl x509 -inform DER -in AC_Raiz_FNMT-RCM_SHA256.cer -out certificado_fnmt.pem
```

Figura 2.3: Conversión del certificado raíz FNMT de DER a PEM.

2.2.4. Generación del fichero de CAs confiables

A continuación, se concatena el certificado de la FNMT con el certificado de la autoridad de la asignatura (certificadoRaiz.crt) en un único fichero PEM que Apache utilizará como almacén de CAs confiables:

```
1 cat certificado_fnmt.pem certificadoRaiz.crt \  
2 > ../Salidas/certificados_confiables.pem
```



```
Nicolás Rey Alonso-[~/Desktop/Practicas_Seguridad_De_La_Informacion/Practica7/ArchivosOriginales]  
$ cat certificado_fnmt.pem certificadoRaiz.crt > ../Salidas/certificados_confiables.pem
```

Figura 2.4: Concatenación del certificado FNMT y de la CA de la asignatura.

El fichero resultante contiene los dos bloques BEGIN CERTIFICATE/END CERTIFICATE, uno por cada CA reconocida.

```

WHFctPVrbtQBULgTfmxKo0nRIBnuvMapGGWn3v7v3QqQIecaZ5JCEJhfTzC8PhxF
tBDXaEaUwED653cXeuYLj2VbPNmaUtu1vZ5Gzz3rkQUcWJaydkxNEJY7kvqcfw+Z
374jNUUeAlz+taibmSxaXvMiwzn15Cou08YfxGyqxRxaQVKL9LFwag0Jl1mpdIC
IfkYtwb1TplvqKtMuejPUBjFd8g5CSxJkjKZqLsXF3mwWsXmo8RZZUc1g16p6DUL
mbvkzSDGm0oG0bVo/CK67lWMK07q87Hj/LaZmtVC+nFNCM+HHmpxfFnTtOmlcYF7
wk5HlQx2doWjKI/pgG6BU6VtX7hI+cL5NqYuSf+4lsKMB70biFj86xsc3i1w4peS
MKGJ47xVqCfWS+2QrYv6YyVZLag13cqXM7zLzced0ezvXg5KKAYmY6252TUtB7p2
ZSysV4999AeU14ECLl2jB0nVetBX+RvnU0Z1qrB5QstocQjpYL05ac70r8NWQMet
UqIJ5G+GR4of6ygnXYMgRwTJbFaai0b1AgMBAAGjgYMWgYAwDwYDVR0TAQH/BAUw
AwEB/zA0BgNVHQ8BAf8EBAMCAQYwHQYDVR00BBYEFpD9xf3E6Jobd2Sn9R2gzL+H
YJptMD4GA1UdIAQ3MDUwMwYEVROgADArMCKGCCsGAQUFBwIBFh1odHRwOi8vd3d3
LmNlcnQum5tdC5lcy9kcGNzLzANBgkqhkiG9w0BAQsFAA0CAgEAB5BK3/MjTvDD
nFFlM5wiooMhfNzKWtN/gHiqQxjAb8EZ6WdmF/9ARP67Jpi6Yb+tmLSbkyU+8B1
RXxLDPIyN8+sD8+Nb/kZ94/sHvJwnvDKu0+3/3Y3dlv2bojzr2IyIpMNM0mqOFGYM
LVN0V2Ue1bLdI4E7pWYjJ2cJj+F3qkPNZVEI7VFY/uY5+ctHhKQV8Xa7p06k08Rf
77IzlhEYt8llvhjho6Tc+hj507wTmzl6NLRtQfv6MooqtyuGC2mDOL7Nii4LcK2N
JpLUHvUBKwrZ1pebbuCoGRw6IYsMHKctA+fdZn71uSANA+iW+YJF1DngoABd15jm
fZ5nc80aKveri6E6F080vFIOiZiaBECEHX5FaNXzuv0+FB8TxxuBE0b+dY7Ixjp
6o7RTUaV8tkasq6+y03m/qZASlaWfot4/nUbQ4mrcFuNLwy+AwF+mWj2z3gyLp
1txyM/1d8iC9djwj2ij3+RvrWWTv3F9yfiD8zYm1kGdNYno/Tq0dwzn+evQoFt9B
9kiABdcPUXmsEKvU7ANm5mqwujGSQkBgjvTcuFqN1W8rB2Vt2lh8k0RdOag0wok
RqEiR9baRRmW1FMDw4R58MD3R++Lj8UGrp1MYp3/RgT408m2ECVAdf4WqsLKYIYv
uu8wd+RU4riEmViAqh0LUTpPSPaLtrM=
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
MIIeAZCCAusCFHxge6KouU3jHyJGgu/ZsLCAKfm2MA0GCSqGSIb3DQEBCwUAMIG9
MQswCQYDVQQGEwJFUzETMBEGA1UECAwKTGFzIFBhbG1hc2EjMCEGA1UEBwwaTGFz
IFBhbG1hc2ZSBHcmFuIENhbmFyaWExDjAMBGNVBAoMBVVMUEEdDMQwwCgYDVQQL
DANFbSUKxLzAtBgNVBAMMJkFzaWduYXR1cmEgU2VndXJpZGFkIGRlIGxhIEluZm9y
bWFiaw9uMSUwIwYJKoZIhvcNAQkBFhZpbmZvQ69wZW5zc2wuy2ljZWkuY29tMB4X
DTIzMDIyODAwMzU0MlloXDTQzMDIyMzA5MzU0MlloZG9wCzA0BgNVBAYTAkVTMRMw
EQYDVQIDApMYXMGUGFsbnWFzMSMwIQYDVQIDBpMYXMGUGFsbnWFzIGRlIEdyYW4g
Q2FuYXJpYTE0MAwGA1UECgwFVUxQR0MxDDAKBgNVBAsMA0VJSTEvMC0GA1UEAwMm
QXNpZ25hdHVVYySBTZWd1cmllkYwQgZGUGbGEGSW5mb3JtYWNpb24xJTAjBgkqhkiG
9w0BCQEWFmluZm9Ab3B1bnNzbC5jaWNlaS5jb20wggeiMA0GCSqGSIb3DQEBAQUA
A4IBDwAwggEKAoIBAQAQDgNHLpTIW6K1s/l3w5ExEhoYGtSMnKAMP9u0+++SYvmcLM
PWL4whNRG/SNoAiQb6MQ+D/N8qlgPT0KSUuPiifJfrPkyVZXZiCRTQajRWq7T5wz
EPA1lGndbE36CDh97eqZSdtrWHjOM3iWJNW/jgH6+RkLIxnZA0mlHqC58WWGyBjQ
zqIm/dIXDhgYNCVZUyd840L3vAM1afRwGImbEtwIttYBxkIOhIpXT+/dijKffK1y
mRgn1cZ3JWZAsM4JevsEd4ZNZBr29qMwQT4kjq+dYBTrGfjYhm2rqFnHP60tUvze
t3jaULqKKJ59oUUF6eM0GkGUb6PwKMqTO+aX+h+LAGMBAEEwDQYJKoZIhvcNAQEL
BQADggEBAI8F+oAf68Gv7+e9uuPAiD5PRbkjNmCuDx4gyHyn01BkClQAY1TtLiCJ
vR0f2ZXk0R0Eah33vHz+Do5cf5KnmFVtFlgQSmn1HLSSL0V+xmLaKAUy7L4S/esy
+DD0VFxKNafVkt6cU2SUu2vaZTGT+JU99lJT2t6G5i7MVYtImkCgvZEvvCFU51j2
rZ/DEONydmjLJfLL2wQuGzC0k7QPNmsm+f3TC+Stz1pA1kkvTzxiGpSiDAY/W9d
IYa0VkozisDGalQYsuDIgcQ/eB29WX9FrC1sjt8k90uRj3n2bi69BVok8aucpERY
5K0bVyrCettoaP67iKN3JR6uoLe6Lm0=
-----END CERTIFICATE-----

```

Figura 2.5: Vista parcial del fichero certificados_confiables.pem.

Capítulo 3

Configuración del servidor Apache

3.1. Enunciado

Modificación de la configuración del Apache para que solicite certificados de cliente y acepte los firmados por la misma autoridad de la asignatura y por otra autoridad (preferentemente FNMT). Habilitación de la ejecución de scripts CGI-bin (bash), código Python, etc.

3.2. Resolución

A partir de la configuración del `VirtualHost *:443` de la práctica anterior, se modifica el fichero `/etc/apache2/sites-available/default-ssl.conf` para añadir la verificación del certificado de cliente, el uso del nuevo almacén de CAs confiables y la habilitación del directorio `cgi-bin`.

En primer lugar se copia el fichero de CAs confiables al directorio de certificados de Apache:

```
1 sudo cp certificados_confiables.pem /etc/apache2/certs/  
2 sudo gedit /etc/apache2/sites-available/default-ssl.conf
```

```

Nicolás Rey Alonso-[~/Desktop/Practicas_Seguridad_De_La_Informacion/Practica7/Salidas]
$ sudo cp certificados_confiables.pem /etc/apache2/certs/

Nicolás Rey Alonso-[~/Desktop/Practicas_Seguridad_De_La_Informacion/Practica7/Salidas]
$ sudo gedit /etc/apache2/sites-available/default-ssl.conf &
[1] 31014

Nicolás Rey Alonso-[~/Desktop/Practicas_Seguridad_De_La_Informacion/Practica7/Salidas]
$
** (gedit:31018): WARNING **: 12:56:45.898: Could not load Peas repository: Typelib file for
** (gedit:31018): WARNING **: 12:56:45.899: Could not load PeasGtk repository: Typelib file
[1] + done      sudo gedit /etc/apache2/sites-available/default-ssl.conf
Nicolás Rey Alonso-[~/Desktop/Practicas_Seguridad_De_La_Informacion/Practica7/Salidas]
$ ls
certificados_confiables.pem

Nicolás Rey Alonso-[~/Desktop/Practicas_Seguridad_De_La_Informacion/Practica7/Salidas]
$ cd /usr/lib/cgi-bin

Nicolás Rey Alonso-[/usr/lib/cgi-bin]
$ ls

Nicolás Rey Alonso-[/usr/lib/cgi-bin]
$ vim index.sh

Nicolás Rey Alonso-[/usr/lib/cgi-bin]
$ sudo vim index.sh

```

Figura 3.1: Copia del fichero de CAs confiables y edición de la configuración de Apache.

El contenido del fichero `default-ssl.conf` queda configurado de la siguiente forma:

```

1 <VirtualHost *:443>
2     ServerName      www.ejemplo.com
3     ServerAlias     ejemplo.com
4     ServerAdmin     nicolasreyalonso@gmail.com
5     DocumentRoot    /var/www/html
6     DirectoryIndex  /cgi-bin/index.sh
7     AddHandler      cgi-script .sh .cgi .pl .py
8     SSLEngine       on
9     SSLCertificateFile /etc/apache2/certs/servidor.crt
10    SSLCertificateKeyFile /etc/apache2/certs/servidor.key
11    SSLCACertificateFile /etc/apache2/certs/certificados_confiables.
    ↪ pem
12    SSLVerifyClient   require
13    SSLVerifyDepth    2
14    SSLProtocol       all -SSLv3 -TLSv1 -TLSv1.1
15    SSLCipherSuite    HIGH:!aNULL:!MD5
16    ErrorLog          ${APACHE_LOG_DIR}/error_ssl.log
17    CustomLog         ${APACHE_LOG_DIR}/access_ssl.log combined
18    ScriptAlias       /cgi-bin/ /usr/lib/cgi-bin/
19    <Directory "/usr/lib/cgi-bin">

```

```
20     AllowOverride None
21     Options +ExecCGI
22     SSLOptions +StdEnvVars
23     Require all granted
24 </Directory>
25 </VirtualHost>
```

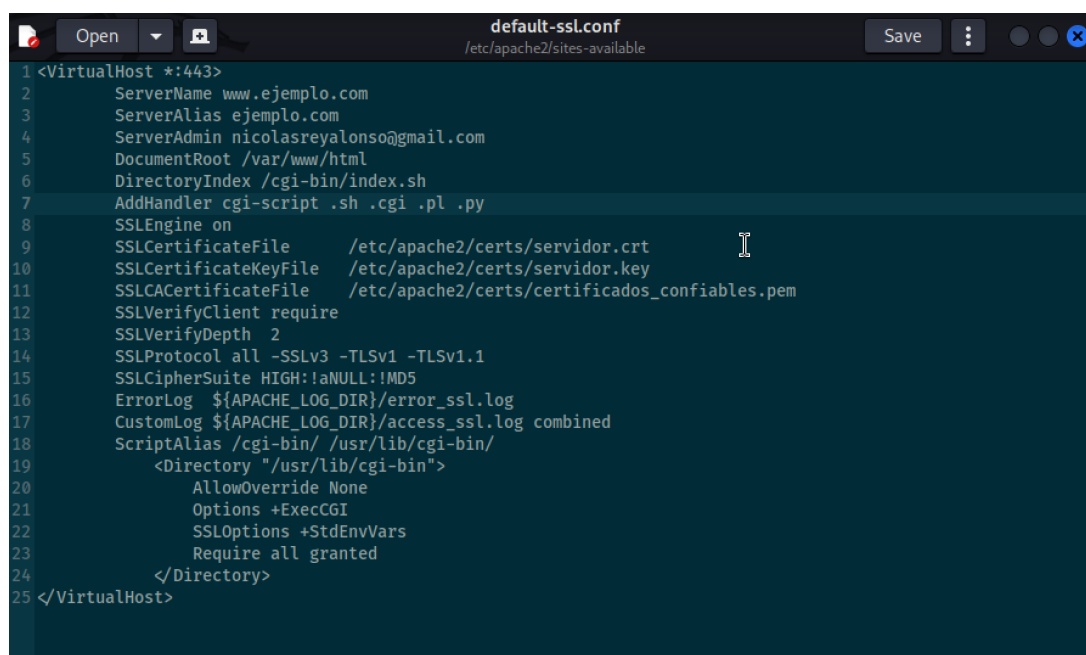


Figura 3.2: Contenido final del fichero default-ssl.conf.

Las directivas más relevantes son:

- `DirectoryIndex /cgi-bin/index.sh`: el script CGI sustituye al `index.html` como página por defecto.
- `AddHandler cgi-script .sh .cgi .pl .py`: habilita la ejecución como CGI de scripts en bash, Perl, Python, etc.
- `SSLCACertificateFile`: apunta al fichero que concatena las CAs (FNMT y asignatura).
- `SSLVerifyClient require`: obliga al cliente a presentar un certificado válido firmado por alguna de las CAs confiables.
- `SSLVerifyDepth 2`: profundidad máxima de la cadena de verificación.
- `SSLOptions +StdEnvVars`: expone los datos del certificado del cliente como variables de entorno (`SSL_CLIENT_S_DN`, `SSL_CLIENT_I_DN`, etc.) para que el script CGI pueda consultarlos sin invocar a `openssl`.

Capítulo 4

Instalación del script de sustitución de `index.html`

4.1. Enunciado

Instalación del script de sustitución de `index.html`. Breve explicación del script utilizado y comprobación de sus permisos.

4.2. Resolución

Se crea el script `/usr/lib/cgi-bin/index.sh`, que sustituye al `index.html` y muestra los datos del certificado de cliente leyéndolos directamente de las variables de entorno que Apache expone gracias a la directiva `SSLOptions +StdEnvVars`.

```
1 #!/bin/bash
2 # Cabecera HTTP obligatoria
3 echo "Content-type: text/html"
4 echo ""
5
6 # Cuerpo de la pagina web
7 echo "<!DOCTYPE html>"
8 echo "<html><head><title>Datos del Certificado</title></head><body>"
9 echo "<h1>Autenticacion de Cliente Exitosa</h1>"
10 echo "<ul>"
11 echo "<li><strong>Sujeto (A quien pertenece):</strong>"
12     ↪ $SSL_CLIENT_S_DN</li>"
13 echo "<li><strong>Emisor (Quien lo firma):</strong> $SSL_CLIENT_I_DN"
```

```

↵ </li>"
13 echo "<li><strong>Validez (Inicio):</strong> $SSL_CLIENT_V_START</li>"
↵ "
14 echo "<li><strong>Validez (Fin):</strong> $SSL_CLIENT_V_END</li>"
15 echo "</ul>"
16 echo "</body></html>"

```

```

#!/bin/bash
# Cabecera HTTP obligatoria
echo "Content-type: text/html"
echo ""

# Cuerpo de la página web
echo "<!DOCTYPE html>"
echo "<html><head><title>Datos del Certificado</title></head><body>"
echo "<h1>Autenticación de Cliente Exitosa</h1>"
echo "<ul>"
echo "<li><strong>Sujeto (A quién pertenece):</strong> $SSL_CLIENT_S_DN</li>"
echo "<li><strong>Emisor (Quién lo firma):</strong> $SSL_CLIENT_I_DN</li>"
echo "<li><strong>Validez (Inicio):</strong> $SSL_CLIENT_V_START</li>"
echo "<li><strong>Validez (Fin):</strong> $SSL_CLIENT_V_END</li>"
echo "</ul>"
echo "</body></html>"

```

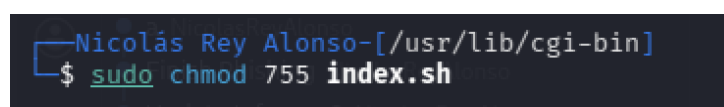
Figura 4.1: Contenido del script CGI `index.sh`.

Funcionamiento del script:

- Imprime la cabecera `HTTP Content-type: text/html` seguida de una línea en blanco, requisito imprescindible de la especificación CGI.
- Construye una página HTML mínima a base de comandos `echo`.
- Lee del entorno las variables `SSL_CLIENT_S_DN` (sujeto del certificado del cliente), `SSL_CLIENT_I_DN` (emisor / CA que lo firma), `SSL_CLIENT_V_START` y `SSL_CLIENT_V_END` (período de validez).
- **No** invoca al comando `openssl`: los datos del certificado son entregados por Apache directamente al script.

Para que Apache pueda ejecutar el script se ajustan los permisos a 755:

```
1 sudo chmod 755 index.sh
```



```

Nicolás Rey Alonso-[/usr/lib/cgi-bin]
$ sudo chmod 755 index.sh

```

Figura 4.2: Asignación de permisos de ejecución al script CGI.

Capítulo 5

Rearranque del Apache y verificación del `error_log`

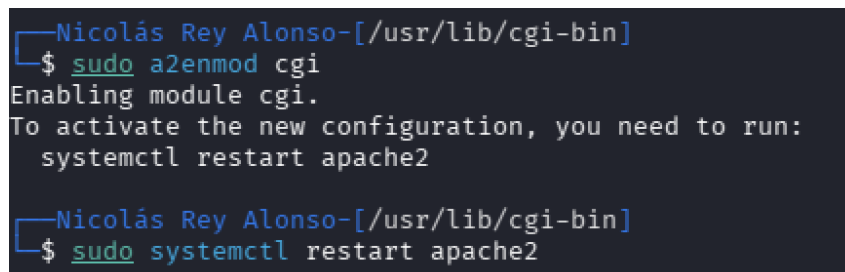
5.1. Enunciado

Rearranque del Apache y verificación del fichero `error_log`.

5.2. Resolución

Antes de reiniciar el servicio se habilita el módulo `cgi` (necesario para que Apache ejecute scripts CGI) y, a continuación, se reinicia el servidor para aplicar los cambios de la configuración:

```
1 sudo a2enmod cgi
2 sudo systemctl restart apache2
```



```
Nicolás Rey Alonso-[/usr/lib/cgi-bin]
$ sudo a2enmod cgi
Enabling module cgi.
To activate the new configuration, you need to run:
    systemctl restart apache2

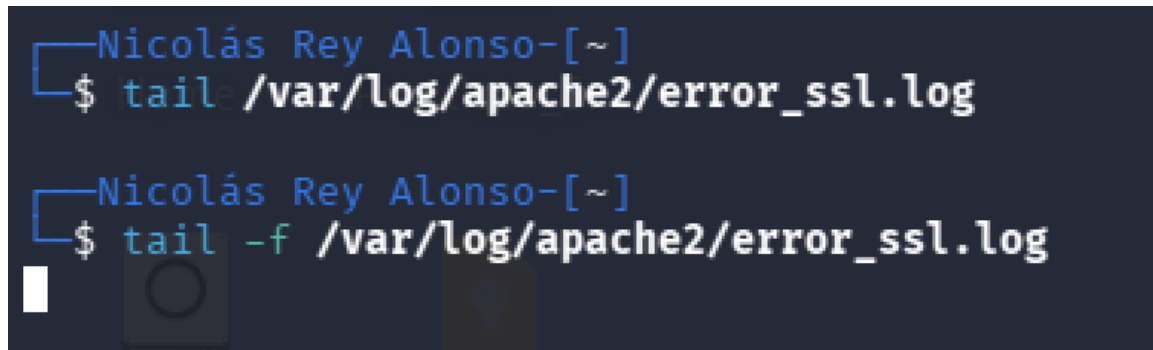
Nicolás Rey Alonso-[/usr/lib/cgi-bin]
$ sudo systemctl restart apache2
```

Figura 5.1: Habilitación del módulo `cgi` y reinicio de Apache.

Tras el reinicio se inspecciona el fichero `/var/log/apache2/error_ssl.log` (definido en la directiva `ErrorLog` del `VirtualHost`) y se verifica que no se han producido

errores ni *warnings*:

```
1 sudo tail -f /var/log/apache2/error_ssl.log
```



```
Nicolás Rey Alonso-[~]  
$ tail /var/log/apache2/error_ssl.log  
  
Nicolás Rey Alonso-[~]  
$ tail -f /var/log/apache2/error_ssl.log
```

Figura 5.2: Inspección del fichero `error_ssl.log`: no se registran errores ni *warnings* tras el reinicio de Apache.

Capítulo 6

Conexión de prueba con openssl s_client

6.1. Enunciado

Conexión de prueba con un comando `openssl s_client` libre de errores. Se emplea un comando del tipo:

```
openssl s_client www.ejemplo.com:443 -cert certcliente.pem -key certclien
```

Se verifica que no se producen errores en la conexión ni en los certificados y que NO hay errores en `error_log`.

6.2. Resolución

Se realiza la conexión empleando el certificado personal del usuario (formato PKCS#12) y el certificado raíz de la asignatura como CA:

```
1 openssl s_client -connect www.ejemplo.com:443 \  
2     -cert /home/nicolo/Desktop/Practicas_Seguridad_De_La_Informacion/  
   ↪ Practica7/Salida/certificadoPersonal.p12 \  
3     -CAfile /etc/apache2/certs/certificados_confiables.pem
```

[illegible]

Figura 6.1: Salida del comando `openssl s_client` con el certificado de cliente.

Se aprecia en la salida la cadena de certificación correcta, con el certificado del servidor firmado por la CA *Asignatura Seguridad de la Información* (ULPGC). El *handshake* TLS finaliza con éxito:

```
SSL handshake has read 4312 bytes and written 3657 bytes
Verification: OK
```

Figura 6.2: Resultado del *handshake* TLS: Verification: OK.

El mensaje `Verification: OK` confirma que el servidor y el cliente se autentican mutuamente sin errores. Una revisión simultánea del fichero `error_ssl.log` confirma que no se ha generado ninguna entrada de error durante la conexión.

Capítulo 7

Conexión desde Firefox con el certificado de la asignatura

7.1. Enunciado

Conexión sin errores desde Firefox con el servidor, empleando el certificado de cliente de la autoridad de la asignatura. El script debe mostrar los datos más importantes del certificado de cliente.

7.2. Resolución

Inicialmente, antes de habilitar la verificación de cliente y el script CGI, al acceder a <https://www.ejemplo.com> se mostraba la página HTML estática creada en la Práctica 6:



Figura 7.1: Página estática original de la Práctica 6.

Tras aplicar la nueva configuración, al volver a cargar la URL Firefox detecta la directiva `SSLVerifyClient require` y solicita al usuario que seleccione el certificado de cliente con el que desea identificarse. Se selecciona el certificado `Nicolas Rey Alonso` firmado por la autoridad de la asignatura.

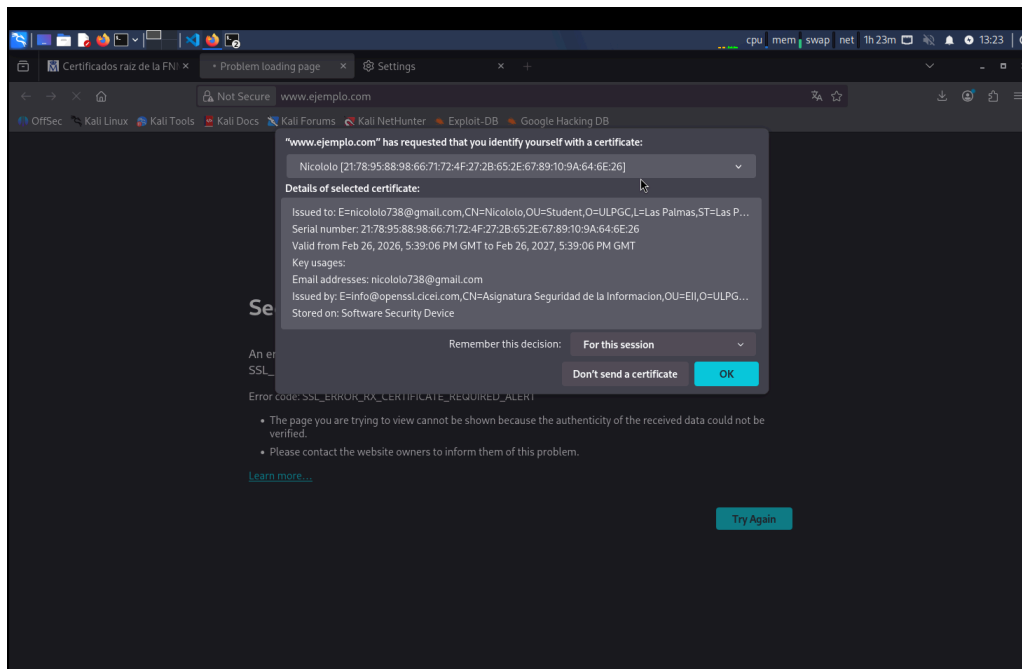


Figura 7.2: Diálogo de selección de certificado de cliente en Firefox.

Una vez aceptado el certificado, Apache exporta sus datos en variables de entorno y ejecuta el script `index.sh`, que genera dinámicamente la siguiente página:

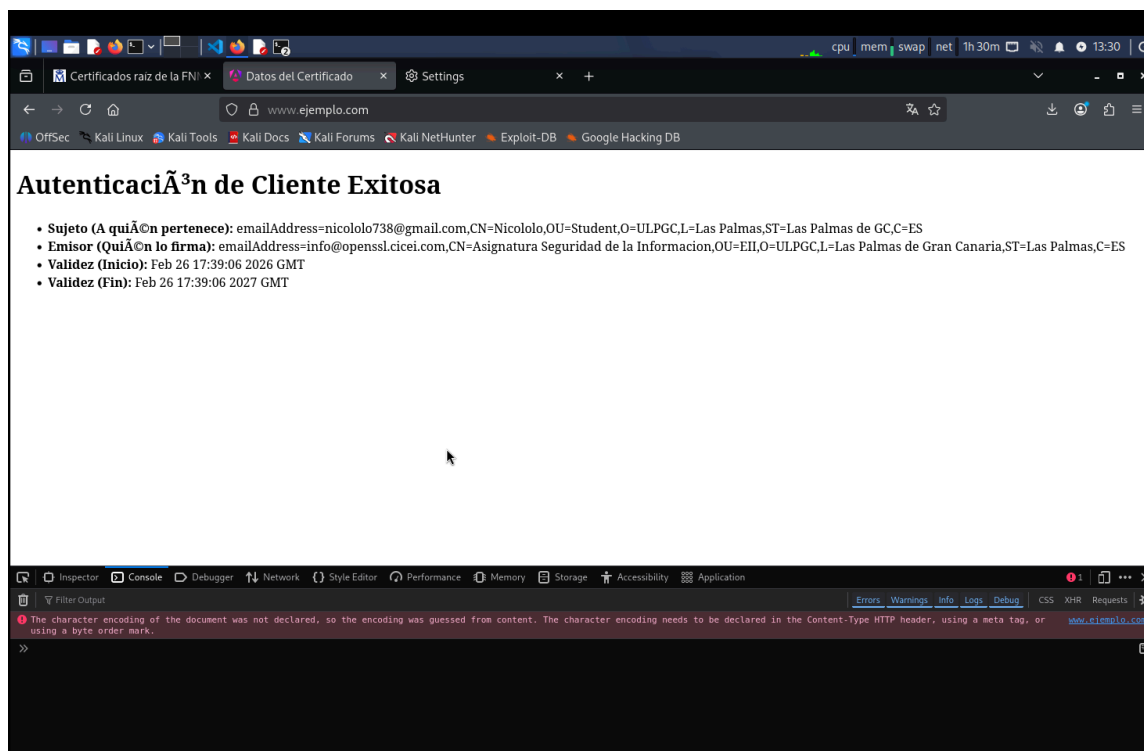


Figura 7.3: Página generada por el script CGI con los datos del certificado de cliente.

Se observa cómo el script muestra correctamente los campos más relevantes del certificado del cliente:

- **Sujeto:** emailAddress=nicololo738@gmail.com, CN=Nicololo, OU=Student, O=ULPGC, L=Las Palmas, ST=Las Palmas de GC, C=ES.
- **Emisor:** emailAddress=info@openssl.cicei.com, CN=Asignatura Seguridad de la Informacion, OU=EII, O=ULPGC, L=Las Palmas de Gran Canaria, ST=Las Palmas, C=ES.
- **Validez (Inicio):** Feb 26 17:39:06 2026 GMT.
- **Validez (Fin):** Feb 26 17:39:06 2027 GMT.

Capítulo 8

Conexión desde Firefox con el segundo certificado de cliente

8.1. Enunciado

Conexión sin errores desde Firefox con el servidor, empleando el certificado de cliente FNMT o uno secundario creado por el estudiante. El script debe mostrar los datos más importantes del certificado de cliente.

8.2. Resolución

Al haber incorporado la **AC Raíz FNMT-RCM** dentro del fichero `certificados_confiables.pem` (véase el Capítulo 2), Apache acepta indistintamente certificados firmados por la autoridad de la asignatura y por la FNMT. Tras instalar en Firefox el correspondiente certificado FNMT del usuario y reiniciar el navegador, al acceder de nuevo a <https://www.ejemplo.com> Firefox vuelve a solicitar el certificado de cliente: en este caso se selecciona el certificado FNMT.

El *handshake* TLS se completa con éxito y el script `index.sh` muestra ahora los datos del nuevo certificado, donde el campo **Emisor** se corresponde con `AC FNMT Usuarios / FNMT-RCM` en lugar de con la autoridad de la asignatura. De esta forma queda demostrado que el servidor es capaz de aceptar dos conexiones distintas firmadas por autoridades de certificación diferentes, tal y como exige el enunciado.

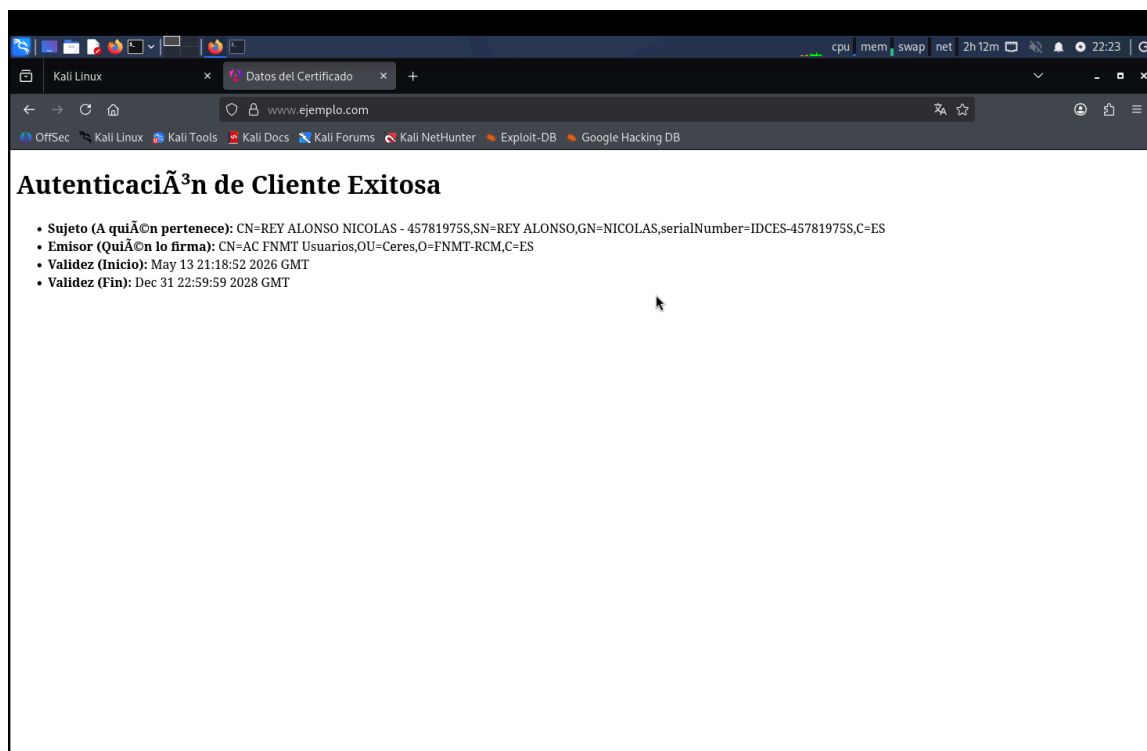


Figura 8.1: Acceso a `www.ejemplo.com` desde Firefox utilizando el certificado de cliente FNMT: el script CGI muestra como **Emisor** la AC FNMT-RCM.

Capítulo 9

Conclusiones

A lo largo de la práctica se ha extendido el servidor web seguro de la Práctica 6 para que, además de presentar su propio certificado de servidor, exija la presentación y verificación de un certificado de cliente, aceptando certificados firmados por dos Autoridades de Certificación distintas: la autoridad propia de la asignatura y la AC Raíz FNMT-RCM.

Los principales logros del trabajo realizado son:

- Descarga, verificación de huella SHA-1 y conversión a PEM del certificado raíz FNMT, así como su concatenación con la CA de la asignatura en un único almacén de CAs confiables.
- Reconfiguración del `VirtualHost *:443` de Apache para activar la verificación obligatoria del certificado del cliente (`SSLVerifyClient require`) y la ejecución de scripts CGI desde `/usr/lib/cgi-bin/`.
- Diseño e instalación de un script `bash` que sustituye al `index.html` estático y muestra los datos más significativos del certificado presentado por el cliente, leyéndolos exclusivamente de las variables de entorno proporcionadas por Apache, sin invocar a `openssl`.
- Validación funcional del servidor mediante `openssl s_client`, obteniendo un *handshake* TLS con `Verification: OK` y sin entradas de error en `error_ssl.log`.
- Comprobación final con Firefox utilizando el certificado de cliente de la asignatura y, posteriormente, un segundo certificado firmado por una autoridad distinta, demostrando así la capacidad del servidor para autenticar clientes mediante múltiples CAs.

En conjunto, la práctica ha permitido consolidar los conceptos de autenticación mutua TLS, gestión de *trust stores* con múltiples CAs y exposición de los metadatos de certificados a aplicaciones CGI a través de variables de entorno, todos ellos elementos fundamentales en el despliegue de servicios web con autenticación fuerte basada en certificados digitales.